Assignment 5: Data Streaming with Apache Kafka

Course: Data Acquisition and Management

Instructor: Muhammad Shahin

PGDM PREDICTIVE ANALYTICS

GURDARSHAN SINGH

Due Date: February 23, 2025

1. Executive Summary

In this assignment, I built a **Weather-App** that streams synthetic weather data for two Canadian cities—**Winnipeg** and **Vancouver**—using **Apache Kafka**. The pipeline consists of:

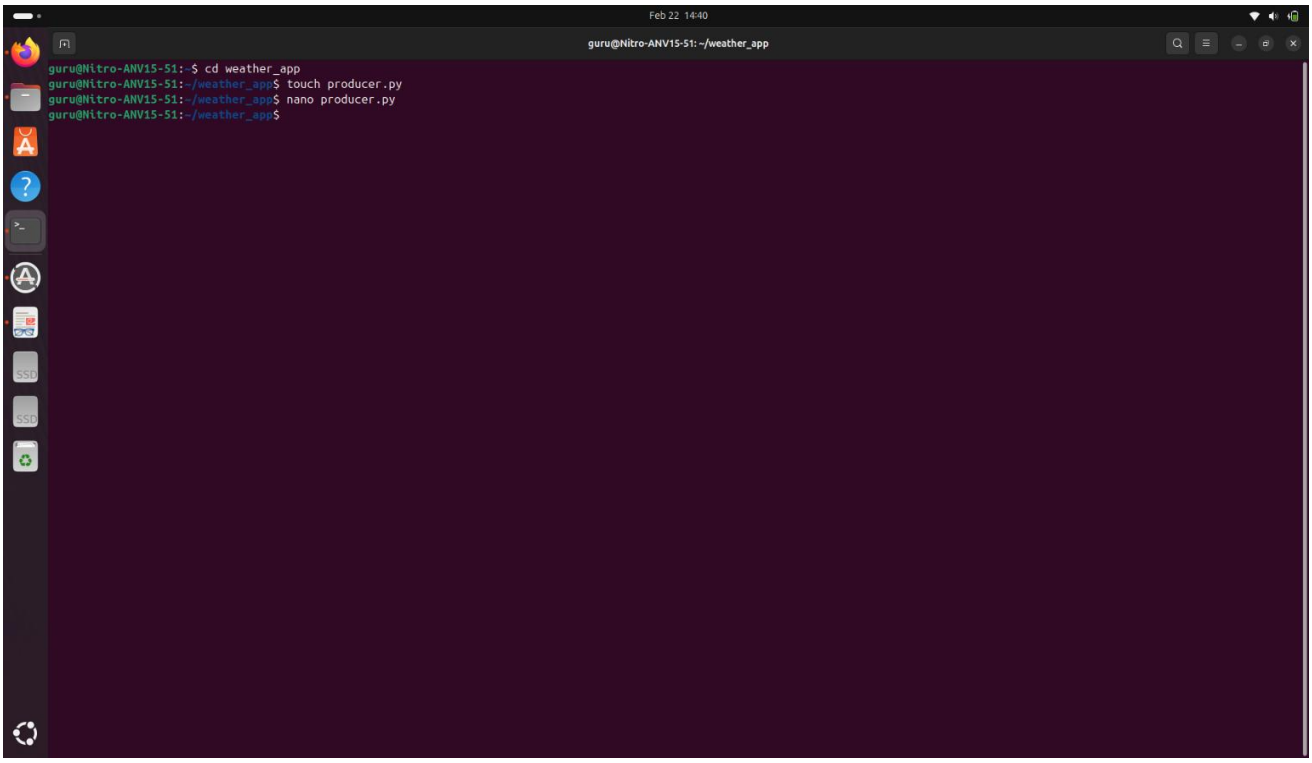
- A **Kafka producer** that sends weather data (temperature, wind speed, humidity) every five seconds to a topic named weather-data.
- A **Kafka consumer** that receives each event, appends a "received_at" timestamp, and inserts it into a **PostgreSQL** database table named weather_events.

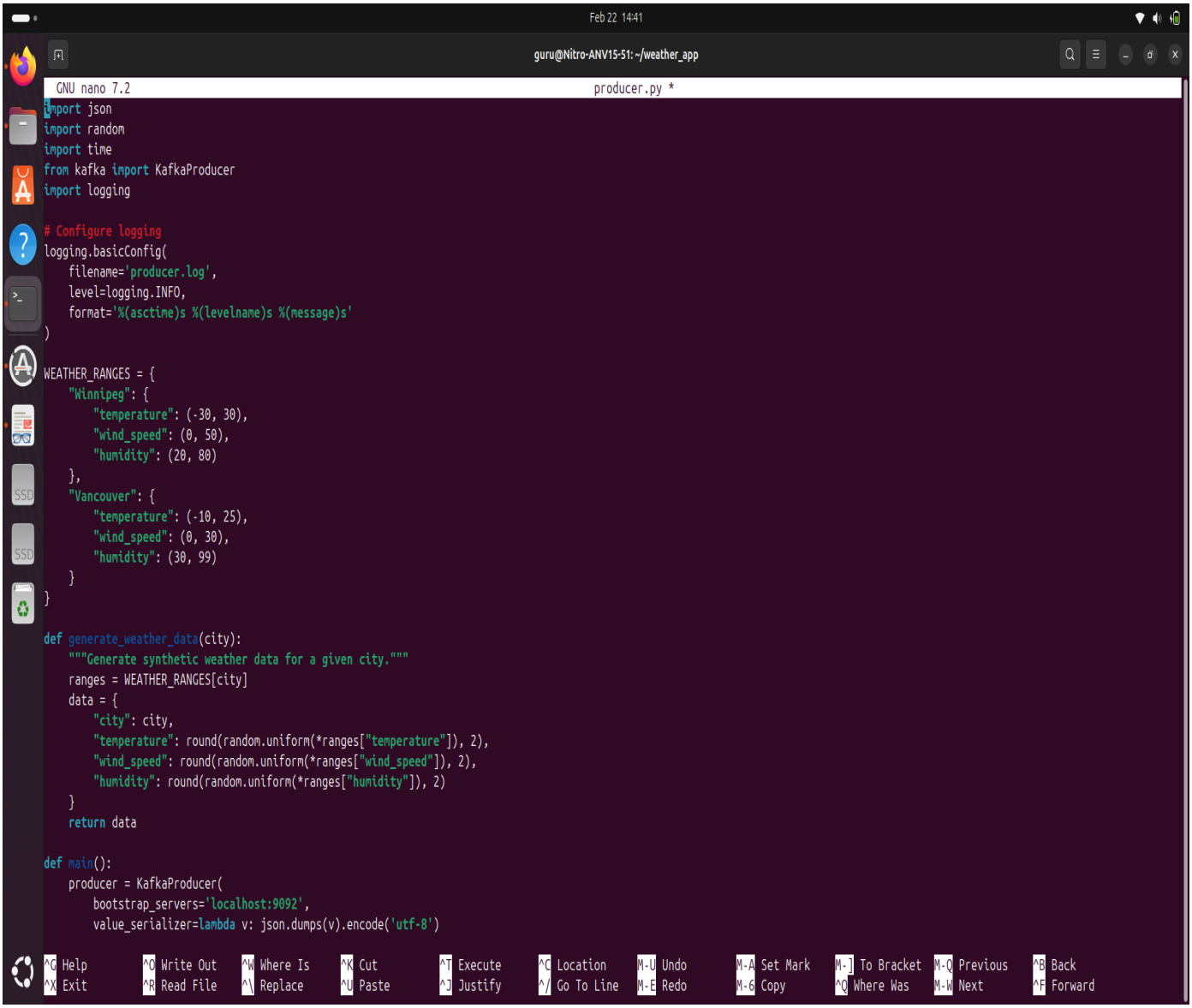
By following the instructions, I verified that data was flowing in real time from the producer to Kafka, then to the consumer, and finally into the database.

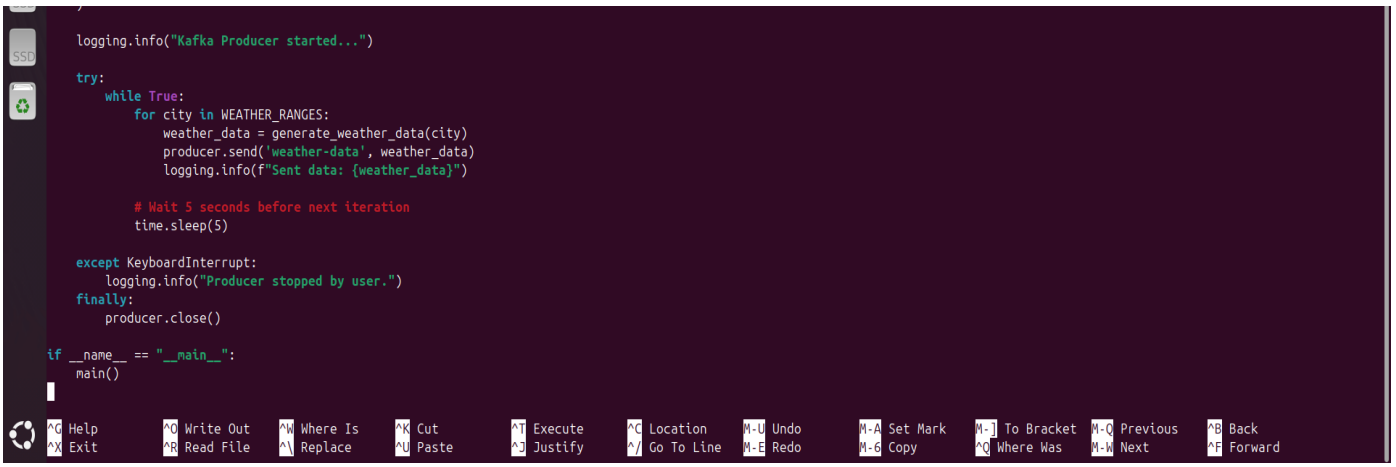
2. Implementation Steps

2.1 Creating the Python Files in the directory weather_app

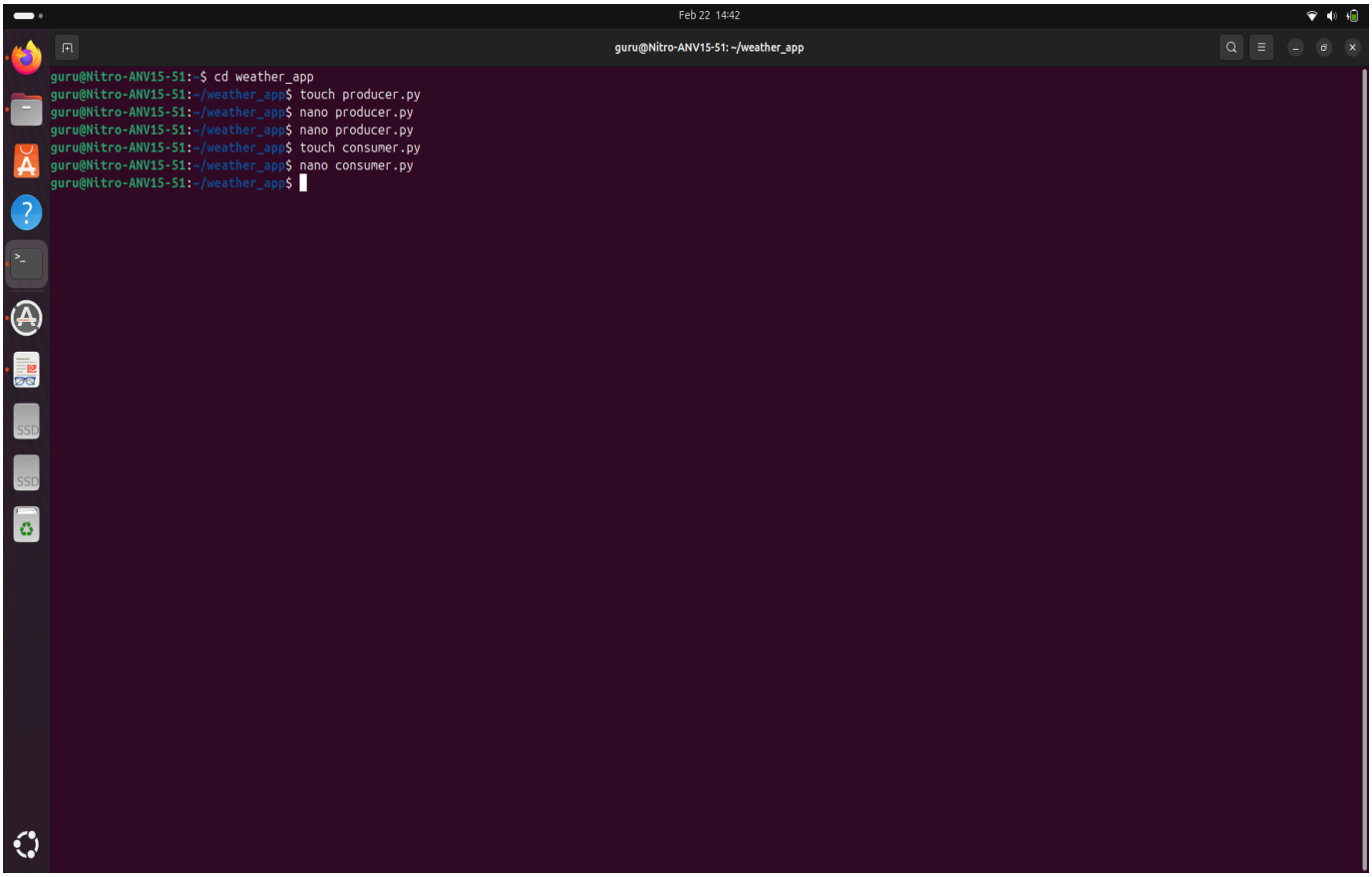
- **Producer Script (producer.py):**
    - Generates synthetic weather data for Winnipeg and Vancouver, each within specified min–max ranges.
    - Sends the data to the Kafka topic (weather-data) in JSON format every five seconds.
    - Logs the messages in a producer log file.

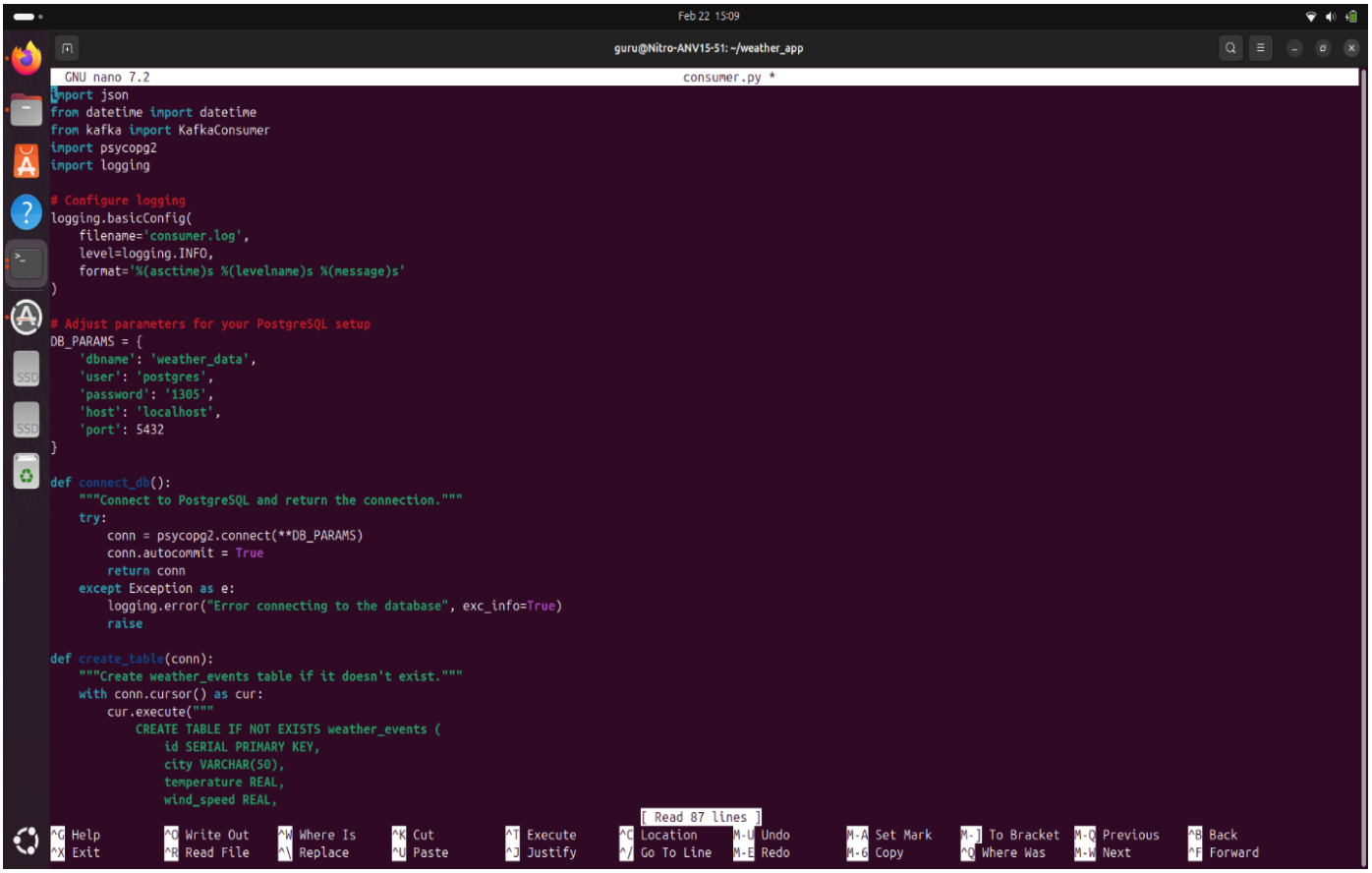


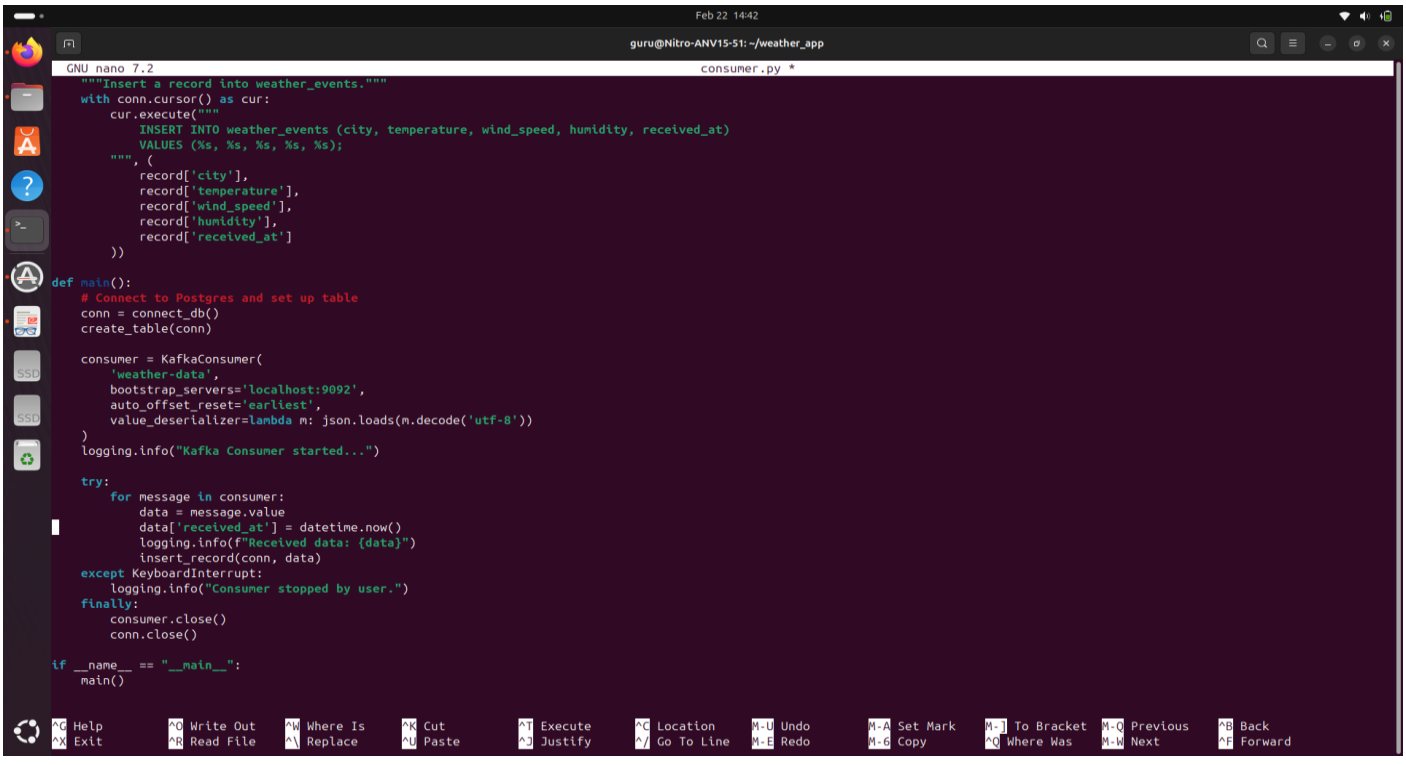

Feb 22 14:41
guru@Nitro-ANV15-51: ~/weather_app
GNU nano 7.2
producer.py *
import json
import random
import time
from kafka import KafkaProducer
import logging
Configure logging
logging.basicConfig(
filename='producer.log',
level=logging.INFO,
format='%(asctime)s %(levelname)s %(message)s'
)
WEATHER_RANGES = {
"Winnipeg": {
"temperature": (-30, 30),
"wind_speed": (0, 50),
"humidity": (20, 80)
},
"Vancouver": {
"temperature": (-10, 25),
"wind_speed": (0, 30),
"humidity": (30, 99)
}
}
def generate_weather_data(city):
"""Generate synthetic weather data for a given city."""
ranges = WEATHER_RANGES[city]
data = {
"city": city,
"temperature": round(random.uniform(*ranges["temperature"]), 2),
"wind_speed": round(random.uniform(*ranges["wind_speed"]), 2),
"humidity": round(random.uniform(*ranges["humidity"]), 2)
}
return data
def main():
producer = KafkaProducer(
bootstrap_servers='localhost:9092',
value_serializer=lambda v: json.dumps(v).encode('utf-8')
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket M-Q Previous ^B Back
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^Q Where Was M-W Next ^F Forward


logging.info("Kafka Producer started...")
try:
while True:
for city in WEATHER_RANGES:
weather_data = generate_weather_data(city)
producer.send('weather-data', weather_data)
logging.info(f"Sent data: {weather_data}")
Wait 5 seconds before next iteration
time.sleep(5)
except KeyboardInterrupt:
logging.info("Producer stopped by user.")
finally:
producer.close()
if __name__ == "__main__":
main()
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket M-Q Previous ^B Back
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^Q Where Was M-W Next ^F Forward

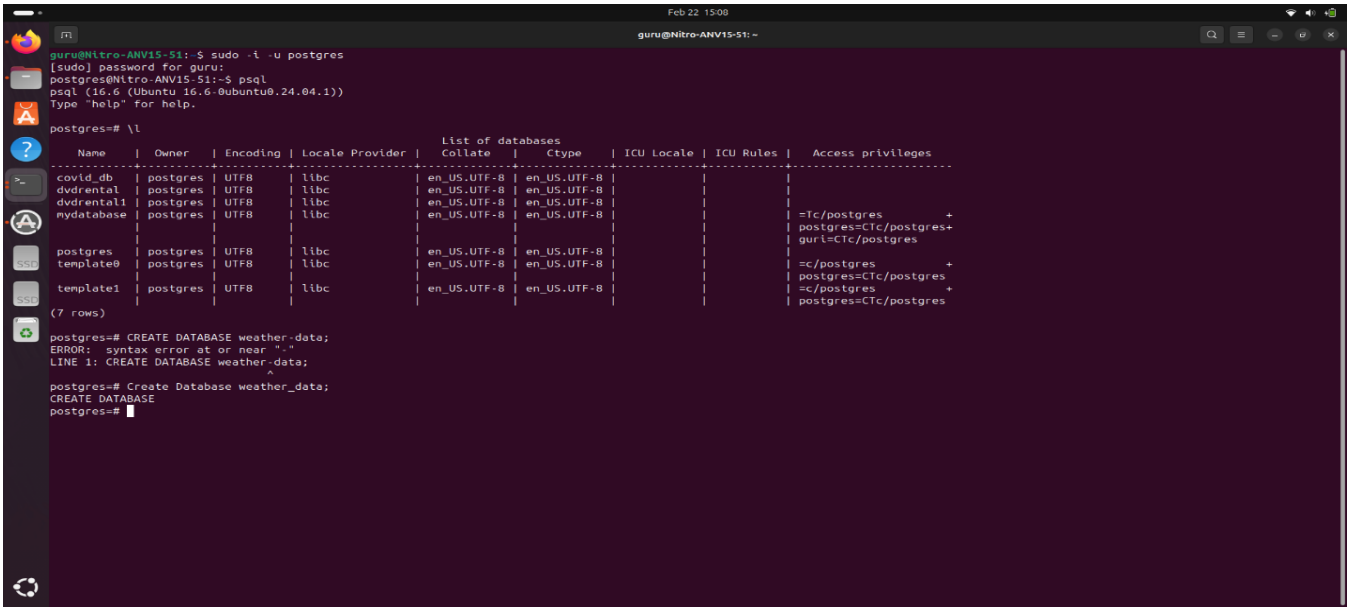
- **Consumer Script (consumer.py):**
    - Subscribes to the weather-data topic.
    - Appends a received_at timestamp to each message.
    - Inserts each message into a Postgres table named weather_events.
    - Logs the consumed records in a consumer log file.




Feb 22 15:09
guru@Nitro-ANV15-51: ~/weather_app
GNU nano 7.2 consumer.py *
import json
from datetime import datetime
from kafka import KafkaConsumer
import psycopg2
import logging
Configure logging
logging.basicConfig(
filename='consumer.log',
level=logging.INFO,
format='%(asctime)s %(levelname)s %(message)s'
)
Adjust parameters for your PostgreSQL setup
DB_PARAMS = {
'dbname': 'weather_data',
'user': 'postgres',
'password': '1305',
'host': 'localhost',
'port': 5432
}
def connect_db():
"""Connect to PostgreSQL and return the connection."""
try:
conn = psycopg2.connect(**DB_PARAMS)
conn.autocommit = True
return conn
except Exception as e:
logging.error("Error connecting to the database", exc_info=True)
raise
def create_table(conn):
"""Create weather_events table if it doesn't exist."""
with conn.cursor() as cur:
cur.execute("""
CREATE TABLE IF NOT EXISTS weather_events (
id SERIAL PRIMARY KEY,
city VARCHAR(50),
temperature REAL,
wind_speed REAL,
[Read 87 lines]
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket M-Q Previous ^B Back
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^Q Where Was M-W Next ^F Forward

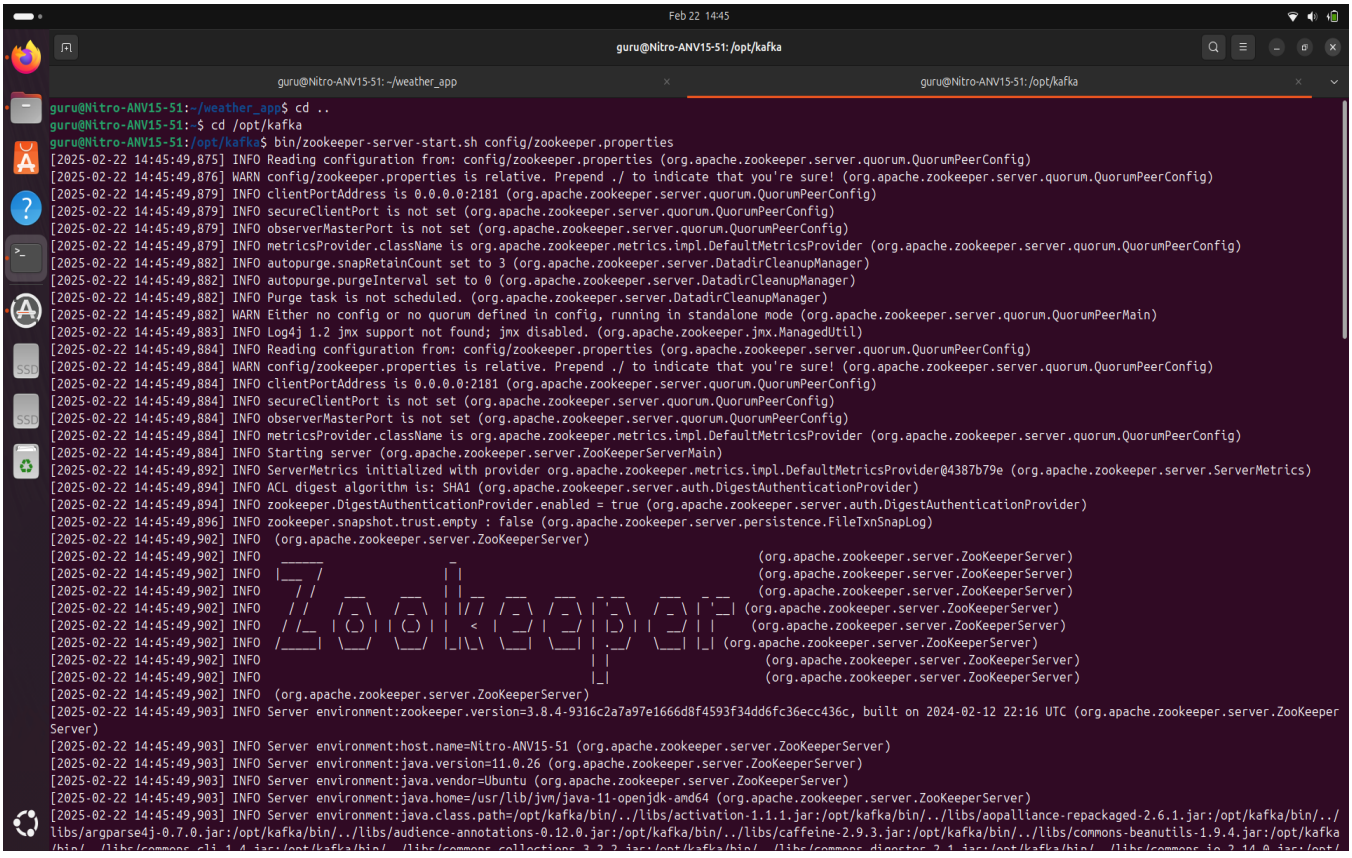

Feb 22 14:42
guru@Nitro-ANV15-51: ~/weather_app
GNU nano 7.2 consumer.py *
"""Insert a record into weather_events."""
with conn.cursor() as cur:
cur.execute("""
INSERT INTO weather_events (city, temperature, wind_speed, humidity, received_at)
VALUES (%s, %s, %s, %s, %s);
""", (
record['city'],
record['temperature'],
record['wind_speed'],
record['humidity'],
record['received_at']
))
def main():
Connect to Postgres and set up table
conn = connect_db()
create_table(conn)
consumer = KafkaConsumer(
'weather-data',
bootstrap_servers='localhost:9092',
auto_offset_reset='earliest',
value_deserializer=lambda m: json.loads(m.decode('utf-8'))
)
logging.info("Kafka Consumer started...")
try:
for message in consumer:
data = message.value
data['received_at'] = datetime.now()
logging.info(f"Received data: {data}")
insert_record(conn, data)
except KeyboardInterrupt:
logging.info("Consumer stopped by user.")
finally:
consumer.close()
conn.close()
if __name__ == "__main__":
main()
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket M-Q Previous ^B Back
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^Q Where Was M-W Next ^F Forward

2.2 Creating a database in postgres of name *weather_data:*



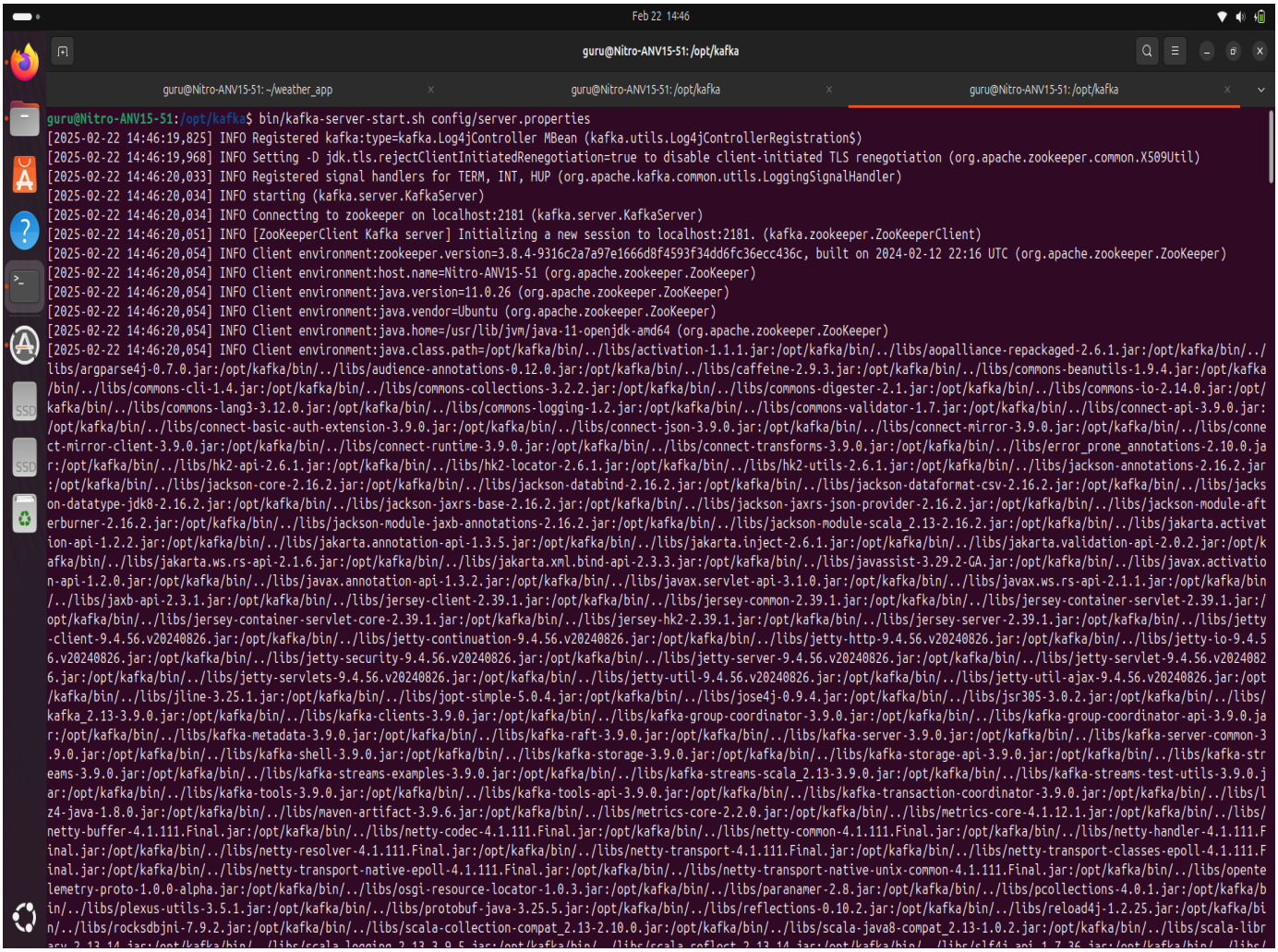
2.3 Start the Zookeeper Service

Once the scripts were ready, I opened the **first terminal** and started the Zookeeper service. I kept that terminal open and running.


Feb 22 14:45
guru@Nitro-ANV15-51: /opt/kafka
guru@Nitro-ANV15-51: ~/weather_app
guru@Nitro-ANV15-51: /opt/kafka
guru@Nitro-ANV15-51:~/weather_app$ cd ..
guru@Nitro-ANV15-51:~$ cd /opt/kafka
guru@Nitro-ANV15-51:/opt/kafka$ bin/zookeeper-server-start.sh config/zookeeper.properties
[2025-02-22 14:45:49,875] INFO Reading configuration from: config/zookeeper.properties (org.apache.zookeeper.server.quorum.QuorumPeerConfig)
[2025-02-22 14:45:49,876] WARN config/zookeeper.properties is relative. Prepend ./ to indicate that you're sure! (org.apache.zookeeper.server.quorum.QuorumPeerConfig)
[2025-02-22 14:45:49,879] INFO clientPortAddress is 0.0.0.0:2181 (org.apache.zookeeper.server.quorum.QuorumPeerConfig)
[2025-02-22 14:45:49,879] INFO secureClientPort is not set (org.apache.zookeeper.server.quorum.QuorumPeerConfig)
[2025-02-22 14:45:49,879] INFO observerMasterPort is not set (org.apache.zookeeper.server.quorum.QuorumPeerConfig)
[2025-02-22 14:45:49,879] INFO metricsProvider.className is org.apache.zookeeper.metrics.impl.DefaultMetricsProvider (org.apache.zookeeper.server.quorum.QuorumPeerConfig)
[2025-02-22 14:45:49,882] INFO autopurge.snapRetainCount set to 3 (org.apache.zookeeper.server.DatadirCleanupManager)
[2025-02-22 14:45:49,882] INFO autopurge.purgeInterval set to 0 (org.apache.zookeeper.server.DatadirCleanupManager)
[2025-02-22 14:45:49,882] INFO Purge task is not scheduled. (org.apache.zookeeper.server.DatadirCleanupManager)
[2025-02-22 14:45:49,882] WARN Either no config or no quorum defined in config, running in standalone mode (org.apache.zookeeper.server.quorum.QuorumPeerMain)
[2025-02-22 14:45:49,883] INFO Log4j 1.2 jmx support not found; jmx disabled. (org.apache.zookeeper.jmx.ManagedUtil)
[2025-02-22 14:45:49,884] INFO Reading configuration from: config/zookeeper.properties (org.apache.zookeeper.server.quorum.QuorumPeerConfig)
[2025-02-22 14:45:49,884] WARN config/zookeeper.properties is relative. Prepend ./ to indicate that you're sure! (org.apache.zookeeper.server.quorum.QuorumPeerConfig)
[2025-02-22 14:45:49,884] INFO clientPortAddress is 0.0.0.0:2181 (org.apache.zookeeper.server.quorum.QuorumPeerConfig)
[2025-02-22 14:45:49,884] INFO secureClientPort is not set (org.apache.zookeeper.server.quorum.QuorumPeerConfig)
[2025-02-22 14:45:49,884] INFO observerMasterPort is not set (org.apache.zookeeper.server.quorum.QuorumPeerConfig)
[2025-02-22 14:45:49,884] INFO metricsProvider.className is org.apache.zookeeper.metrics.impl.DefaultMetricsProvider (org.apache.zookeeper.server.quorum.QuorumPeerConfig)
[2025-02-22 14:45:49,884] INFO Starting server (org.apache.zookeeper.server.ZooKeeperServerMain)
[2025-02-22 14:45:49,892] INFO ServerMetrics initialized with provider org.apache.zookeeper.metrics.impl.DefaultMetricsProvider@4387b79e (org.apache.zookeeper.server.ServerMetrics)
[2025-02-22 14:45:49,894] INFO ACL digest algorithm is: SHA1 (org.apache.zookeeper.server.auth.DigestAuthenticationProvider)
[2025-02-22 14:45:49,894] INFO zookeeper.DigestAuthenticationProvider.enabled = true (org.apache.zookeeper.server.auth.DigestAuthenticationProvider)
[2025-02-22 14:45:49,896] INFO zookeeper.snapshot.trust.empty : false (org.apache.zookeeper.server.persistence.FileTxnSnapLog)
[2025-02-22 14:45:49,902] INFO (org.apache.zookeeper.server.ZooKeeperServer

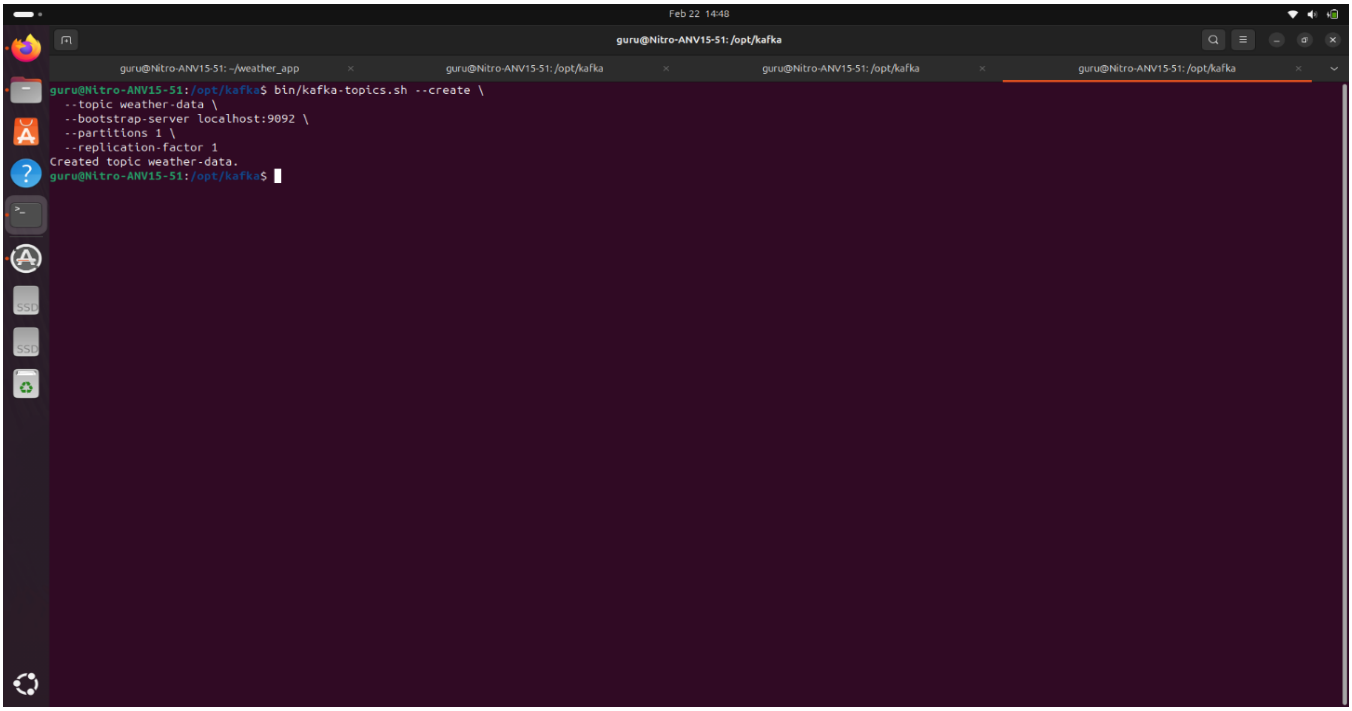
2.4 Start the Kafka Service

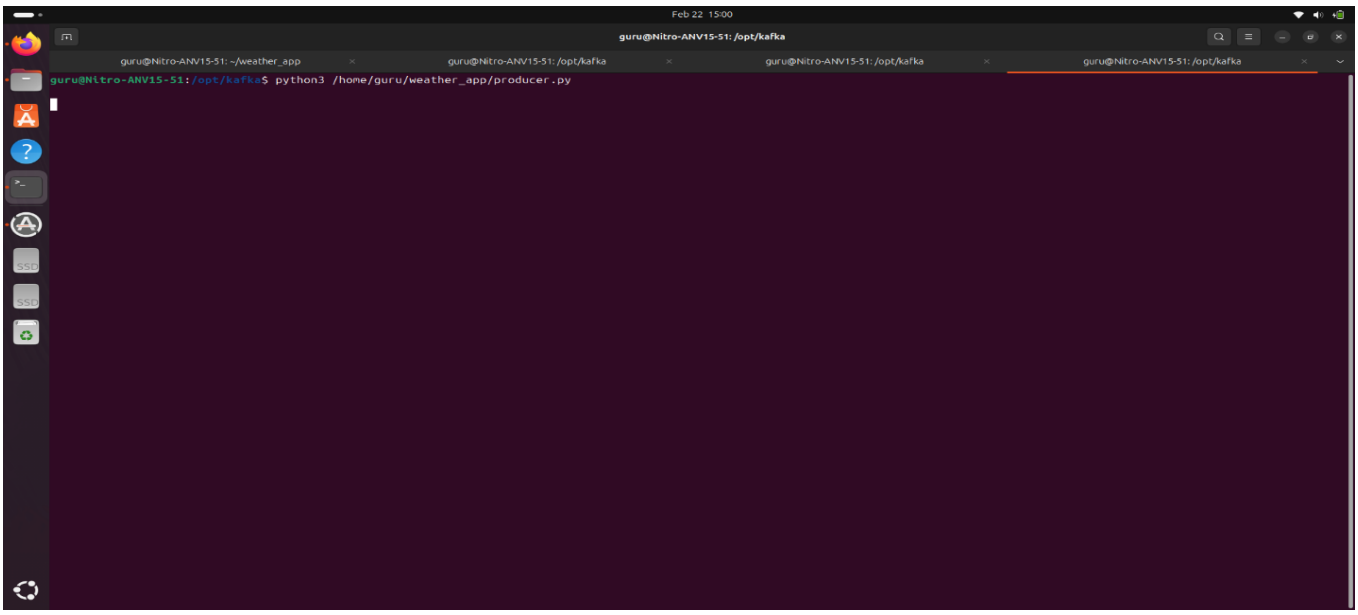
In a **second terminal**, I started the Kafka broker. This terminal also remained open.



2.5 Create the Kafka Topic & Run Producer

1. **Created** the Kafka topic weather-data.
2. **Ran** the producer script, which generated synthetic weather data and sent it to the weather-data topic every five seconds. The producer also logged each message.





2.6 Run the Consumer

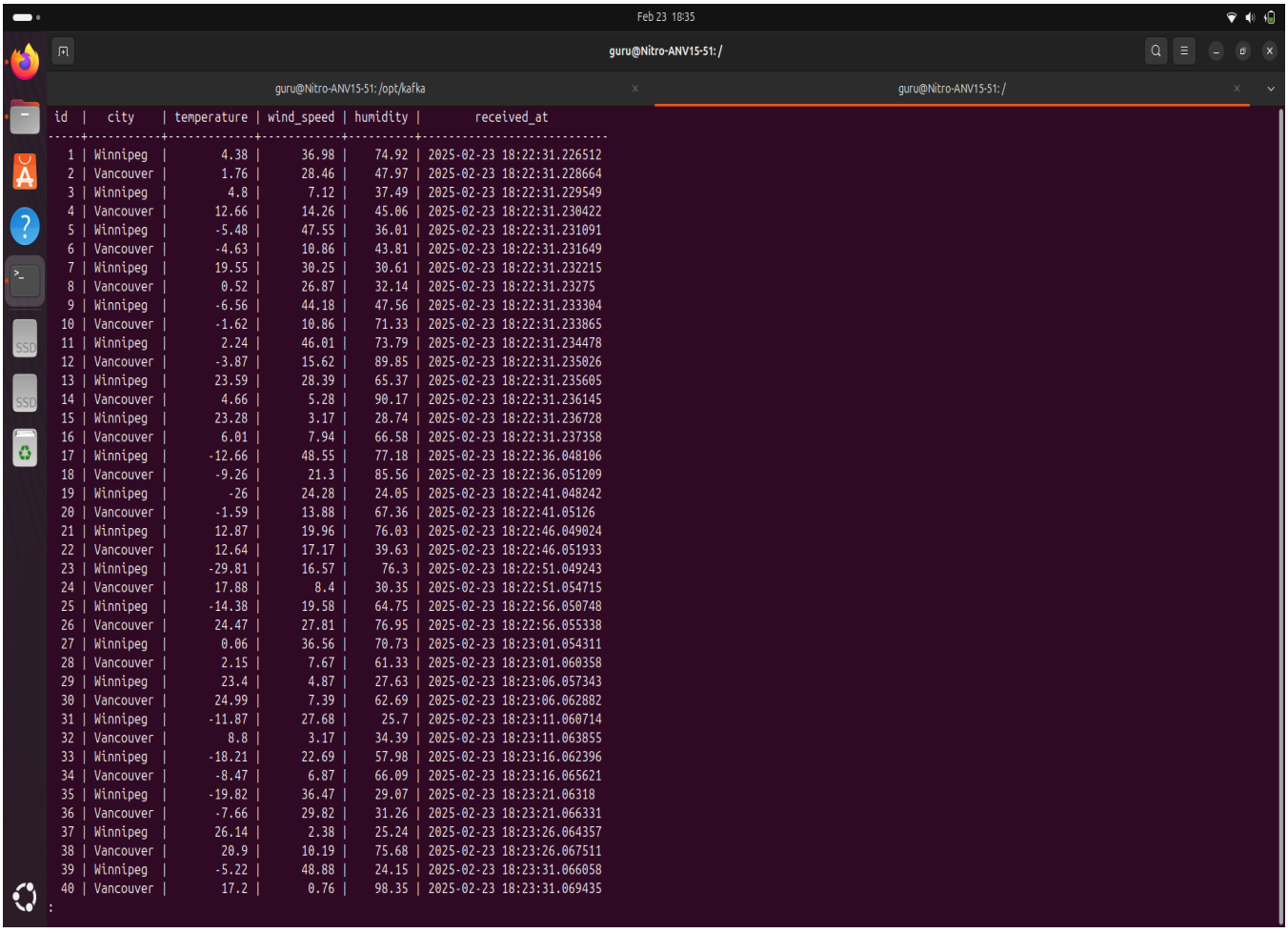
I opened a **fourth terminal** and ran the consumer script. It consumed messages from the weather-data topic, appended timestamps, and inserted records into the weather_events table in Postgres.

2.7 Stop the Producer After 10 minutes

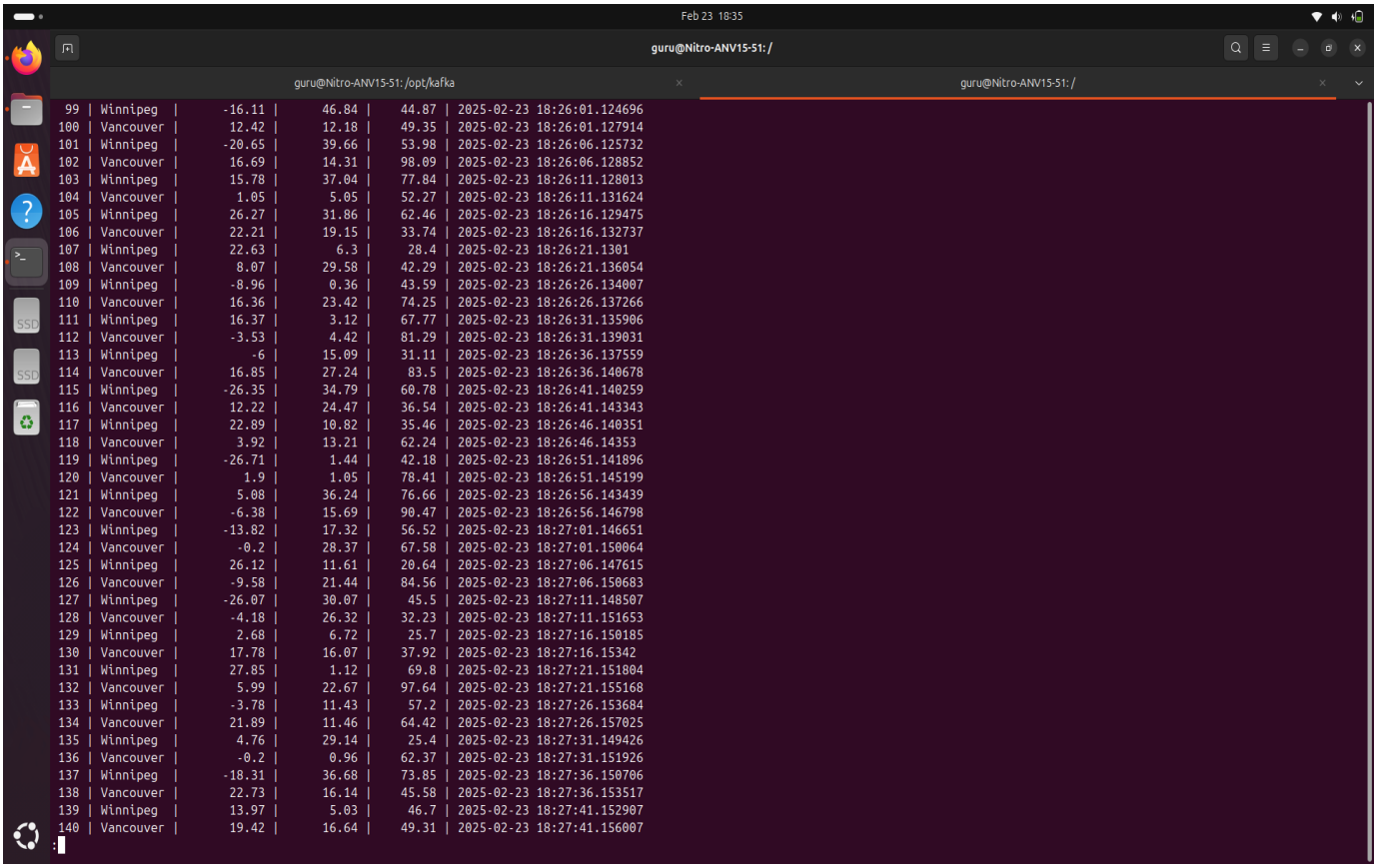
After letting the **producer** run for about 11 to 12 mins, I **stopped** it (Ctrl + C). This ended the continuous flow of weather events.

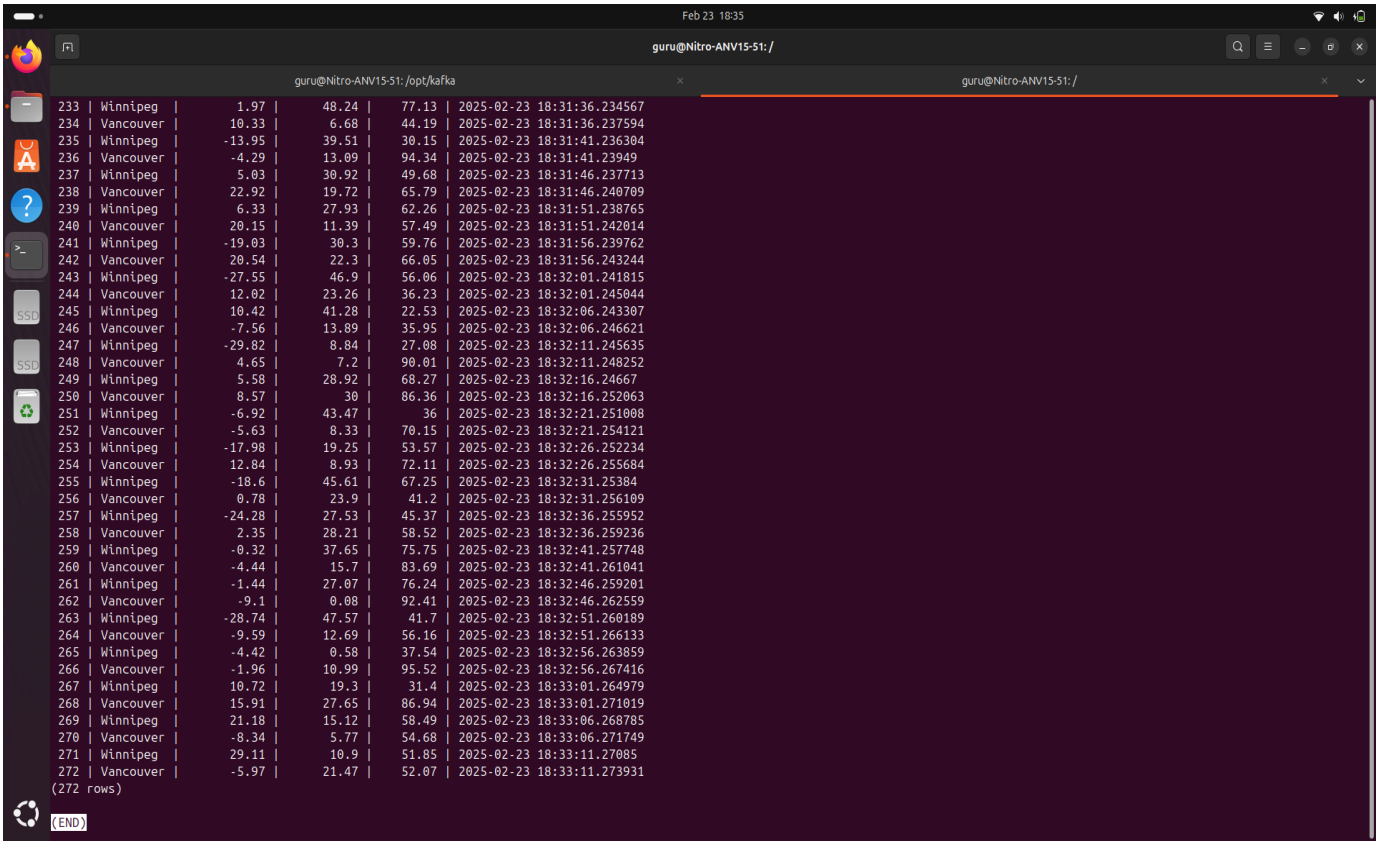
2.8 Verify Data in the Database

Using a Postgres client (psql or similar), I ran a ***SELECT * FROM weather_events;*** query and confirmed that 272 rows were inserted for Winnipeg and Vancouver, each with a “received_at” timestamp.



id	city	temperature	wind_speed	humidity	received_at
1	Winnipeg	4.38	36.98	74.92	2025-02-23 18:22:31.226512
2	Vancouver	1.76	28.46	47.97	2025-02-23 18:22:31.228664
3	Winnipeg	4.8	7.12	37.49	2025-02-23 18:22:31.229549
4	Vancouver	12.66	14.26	45.06	2025-02-23 18:22:31.230422
5	Winnipeg	-5.48	47.55	36.01	2025-02-23 18:22:31.231091
6	Vancouver	-4.63	10.86	43.81	2025-02-23 18:22:31.231649
7	Winnipeg	19.55	30.25	30.61	2025-02-23 18:22:31.232215
8	Vancouver	0.52	26.87	32.14	2025-02-23 18:22:31.23275
9	Winnipeg	-6.56	44.18	47.56	2025-02-23 18:22:31.233304
10	Vancouver	-1.62	10.86	71.33	2025-02-23 18:22:31.233865
11	Winnipeg	2.24	46.01	73.79	2025-02-23 18:22:31.234478
12	Vancouver	-3.87	15.62	89.85	2025-02-23 18:22:31.235026
13	Winnipeg	23.59	28.39	65.37	2025-02-23 18:22:31.235605
14	Vancouver	4.66	5.28	90.17	2025-02-23 18:22:31.236145
15	Winnipeg	23.28	3.17	28.74	2025-02-23 18:22:31.236728
16	Vancouver	6.01	7.94	66.58	2025-02-23 18:22:31.237358
17	Winnipeg	-12.66	48.55	77.18	2025-02-23 18:22:36.048106
18	Vancouver	-9.26	21.3	85.56	2025-02-23 18:22:36.051209
19	Winnipeg	-26	24.28	24.05	2025-02-23 18:22:41.048242
20	Vancouver	-1.59	13.88	67.36	2025-02-23 18:22:41.05126
21	Winnipeg	12.87	19.96	76.03	2025-02-23 18:22:46.049024
22	Vancouver	12.64	17.17	39.63	2025-02-23 18:22:46.051933
23	Winnipeg	-29.81	16.57	76.3	2025-02-23 18:22:51.049243
24	Vancouver	17.88	8.4	30.35	2025-02-23 18:22:51.054715
25	Winnipeg	-14.38	19.58	64.75	2025-02-23 18:22:56.050748
26	Vancouver	24.47	27.81	76.95	2025-02-23 18:22:56.055338
27	Winnipeg	0.06	36.56	70.73	2025-02-23 18:23:01.054311
28	Vancouver	2.15	7.67	61.33	2025-02-23 18:23:01.060358
29	Winnipeg	23.4	4.87	27.63	2025-02-23 18:23:06.057343
30	Vancouver	24.99	7.39	62.69	2025-02-23 18:23:06.062882
31	Winnipeg	-11.87	27.68	25.7	2025-02-23 18:23:11.060714
32	Vancouver	8.8	3.17	34.39	2025-02-23 18:23:11.063855
33	Winnipeg	-18.21	22.69	57.98	2025-02-23 18:23:16.062396
34	Vancouver	-8.47	6.87	66.09	2025-02-23 18:23:16.065621
35	Winnipeg	-19.82	36.47	29.07	2025-02-23 18:23:21.06318
36	Vancouver	-7.66	29.82	31.26	2025-02-23 18:23:21.066331
37	Winnipeg	26.14	2.38	25.24	2025-02-23 18:23:26.064357
38	Vancouver	20.9	10.19	75.68	2025-02-23 18:23:26.067511
39	Winnipeg	-5.22	48.88	24.15	2025-02-23 18:23:31.066058
40	Vancouver	17.2	0.76	98.35	2025-02-23 18:23:31.069435


Feb 23 18:35
guru@Nitro-ANV15-51: /
guru@Nitro-ANV15-51: /opt/kafka
guru@Nitro-ANV15-51: /
99 | Winnipeg | -16.11 | 46.84 | 44.87 | 2025-02-23 18:26:01.124696
100 | Vancouver | 12.42 | 12.18 | 49.35 | 2025-02-23 18:26:01.127914
101 | Winnipeg | -20.65 | 39.66 | 53.98 | 2025-02-23 18:26:06.125732
102 | Vancouver | 16.69 | 14.31 | 98.09 | 2025-02-23 18:26:06.128852
103 | Winnipeg | 15.78 | 37.04 | 77.84 | 2025-02-23 18:26:11.128013
104 | Vancouver | 1.05 | 5.05 | 52.27 | 2025-02-23 18:26:11.131624
105 | Winnipeg | 26.27 | 31.86 | 62.46 | 2025-02-23 18:26:16.129475
106 | Vancouver | 22.21 | 19.15 | 33.74 | 2025-02-23 18:26:16.132737
107 | Winnipeg | 22.63 | 6.3 | 28.4 | 2025-02-23 18:26:21.1301
108 | Vancouver | 8.07 | 29.58 | 42.29 | 2025-02-23 18:26:21.136054
109 | Winnipeg | -8.96 | 0.36 | 43.59 | 2025-02-23 18:26:26.134007
110 | Vancouver | 16.36 | 23.42 | 74.25 | 2025-02-23 18:26:26.137266
111 | Winnipeg | 16.37 | 3.12 | 67.77 | 2025-02-23 18:26:31.135906
112 | Vancouver | -3.53 | 4.42 | 81.29 | 2025-02-23 18:26:31.139031
113 | Winnipeg | -6 | 15.09 | 31.11 | 2025-02-23 18:26:36.137559
114 | Vancouver | 16.85 | 27.24 | 83.5 | 2025-02-23 18:26:36.140678
115 | Winnipeg | -26.35 | 34.79 | 60.78 | 2025-02-23 18:26:41.140259
116 | Vancouver | 12.22 | 24.47 | 36.54 | 2025-02-23 18:26:41.143343
117 | Winnipeg | 22.89 | 10.82 | 35.46 | 2025-02-23 18:26:46.140351
118 | Vancouver | 3.92 | 13.21 | 62.24 | 2025-02-23 18:26:46.14353
119 | Winnipeg | -26.71 | 1.44 | 42.18 | 2025-02-23 18:26:51.141896
120 | Vancouver | 1.9 | 1.05 | 78.41 | 2025-02-23 18:26:51.145199
121 | Winnipeg | 5.08 | 36.24 | 76.66 | 2025-02-23 18:26:56.143439
122 | Vancouver | -6.38 | 15.69 | 90.47 | 2025-02-23 18:26:56.146798
123 | Winnipeg | -13.82 | 17.32 | 56.52 | 2025-02-23 18:27:01.146651
124 | Vancouver | -0.2 | 28.37 | 67.58 | 2025-02-23 18:27:01.150064
125 | Winnipeg | 26.12 | 11.61 | 20.64 | 2025-02-23 18:27:06.147615
126 | Vancouver | -9.58 | 21.44 | 84.56 | 2025-02-23 18:27:06.150683
127 | Winnipeg | -26.07 | 30.07 | 45.5 | 2025-02-23 18:27:11.148507
128 | Vancouver | -4.18 | 26.32 | 32.23 | 2025-02-23 18:27:11.151653
129 | Winnipeg | 2.68 | 6.72 | 25.7 | 2025-02-23 18:27:16.150185
130 | Vancouver | 17.78 | 16.07 | 37.92 | 2025-02-23 18:27:16.15342
131 | Winnipeg | 27.85 | 1.12 | 69.8 | 2025-02-23 18:27:21.151804
132 | Vancouver | 5.99 | 22.67 | 97.64 | 2025-02-23 18:27:21.155168
133 | Winnipeg | -3.78 | 11.43 | 57.2 | 2025-02-23 18:27:26.153684
134 | Vancouver | 21.89 | 11.46 | 64.42 | 2025-02-23 18:27:26.157025
135 | Winnipeg | 4.76 | 29.14 | 25.4 | 2025-02-23 18:27:31.149426
136 | Vancouver | -0.2 | 0.96 | 62.37 | 2025-02-23 18:27:31.151926
137 | Winnipeg | -18.31 | 36.68 | 73.85 | 2025-02-23 18:27:36.150706
138 | Vancouver | 22.73 | 16.14 | 45.58 | 2025-02-23 18:27:36.153517
139 | Winnipeg | 13.97 | 5.03 | 46.7 | 2025-02-23 18:27:41.152907
140 | Vancouver | 19.42 | 16.64 | 49.31 | 2025-02-23 18:27:41.156007


Feb 23 18:35
guru@Nitro-ANV15-51: /
guru@Nitro-ANV15-51: /opt/kafka
guru@Nitro-ANV15-51: /
233 | Winnipeg | 1.97 | 48.24 | 77.13 | 2025-02-23 18:31:36.234567
234 | Vancouver | 10.33 | 6.68 | 44.19 | 2025-02-23 18:31:36.237594
235 | Winnipeg | -13.95 | 39.51 | 30.15 | 2025-02-23 18:31:41.236304
236 | Vancouver | -4.29 | 13.09 | 94.34 | 2025-02-23 18:31:41.23949
237 | Winnipeg | 5.03 | 30.92 | 49.68 | 2025-02-23 18:31:46.237713
238 | Vancouver | 22.92 | 19.72 | 65.79 | 2025-02-23 18:31:46.240709
239 | Winnipeg | 6.33 | 27.93 | 62.26 | 2025-02-23 18:31:51.238765
240 | Vancouver | 20.15 | 11.39 | 57.49 | 2025-02-23 18:31:51.242014
241 | Winnipeg | -19.03 | 30.3 | 59.76 | 2025-02-23 18:31:56.239762
242 | Vancouver | 20.54 | 22.3 | 66.05 | 2025-02-23 18:31:56.243244
243 | Winnipeg | -27.55 | 46.9 | 56.06 | 2025-02-23 18:32:01.241815
244 | Vancouver | 12.02 | 23.26 | 36.23 | 2025-02-23 18:32:01.245044
245 | Winnipeg | 10.42 | 41.28 | 22.53 | 2025-02-23 18:32:06.243307
246 | Vancouver | -7.56 | 13.89 | 35.95 | 2025-02-23 18:32:06.246621
247 | Winnipeg | -29.82 | 8.84 | 27.08 | 2025-02-23 18:32:11.245635
248 | Vancouver | 4.65 | 7.2 | 90.01 | 2025-02-23 18:32:11.248252
249 | Winnipeg | 5.58 | 28.92 | 68.27 | 2025-02-23 18:32:16.24667
250 | Vancouver | 8.57 | 30 | 86.36 | 2025-02-23 18:32:16.252063
251 | Winnipeg | -6.92 | 43.47 | 36 | 2025-02-23 18:32:21.251008
252 | Vancouver | -5.63 | 8.33 | 70.15 | 2025-02-23 18:32:21.254121
253 | Winnipeg | -17.98 | 19.25 | 53.57 | 2025-02-23 18:32:26.252234
254 | Vancouver | 12.84 | 8.93 | 72.11 | 2025-02-23 18:32:26.255684
255 | Winnipeg | -18.6 | 45.61 | 67.25 | 2025-02-23 18:32:31.25384
256 | Vancouver | 0.78 | 23.9 | 41.2 | 2025-02-23 18:32:31.256109
257 | Winnipeg | -24.28 | 27.53 | 45.37 | 2025-02-23 18:32:36.255952
258 | Vancouver | 2.35 | 28.21 | 58.52 | 2025-02-23 18:32:36.259236
259 | Winnipeg | -0.32 | 37.65 | 75.75 | 2025-02-23 18:32:41.257748
260 | Vancouver | -4.44 | 15.7 | 83.69 | 2025-02-23 18:32:41.261041
261 | Winnipeg | -1.44 | 27.07 | 76.24 | 2025-02-23 18:32:46.259201
262 | Vancouver | -9.1 | 0.08 | 92.41 | 2025-02-23 18:32:46.262559
263 | Winnipeg | -28.74 | 47.57 | 41.7 | 2025-02-23 18:32:51.260189
264 | Vancouver | -9.59 | 12.69 | 56.16 | 2025-02-23 18:32:51.266133
265 | Winnipeg | -4.42 | 0.58 | 37.54 | 2025-02-23 18:32:56.263859
266 | Vancouver | -1.96 | 10.99 | 95.52 | 2025-02-23 18:32:56.267416
267 | Winnipeg | 10.72 | 19.3 | 31.4 | 2025-02-23 18:33:01.264979
268 | Vancouver | 15.91 | 27.65 | 86.94 | 2025-02-23 18:33:01.271019
269 | Winnipeg | 21.18 | 15.12 | 58.49 | 2025-02-23 18:33:06.268785
270 | Vancouver | -8.34 | 5.77 | 54.68 | 2025-02-23 18:33:06.271749
271 | Winnipeg | 29.11 | 10.9 | 51.85 | 2025-02-23 18:33:11.27085
272 | Vancouver | -5.97 | 21.47 | 52.07 | 2025-02-23 18:33:11.273931
(272 rows)
(END)

2.9 Stop All Services

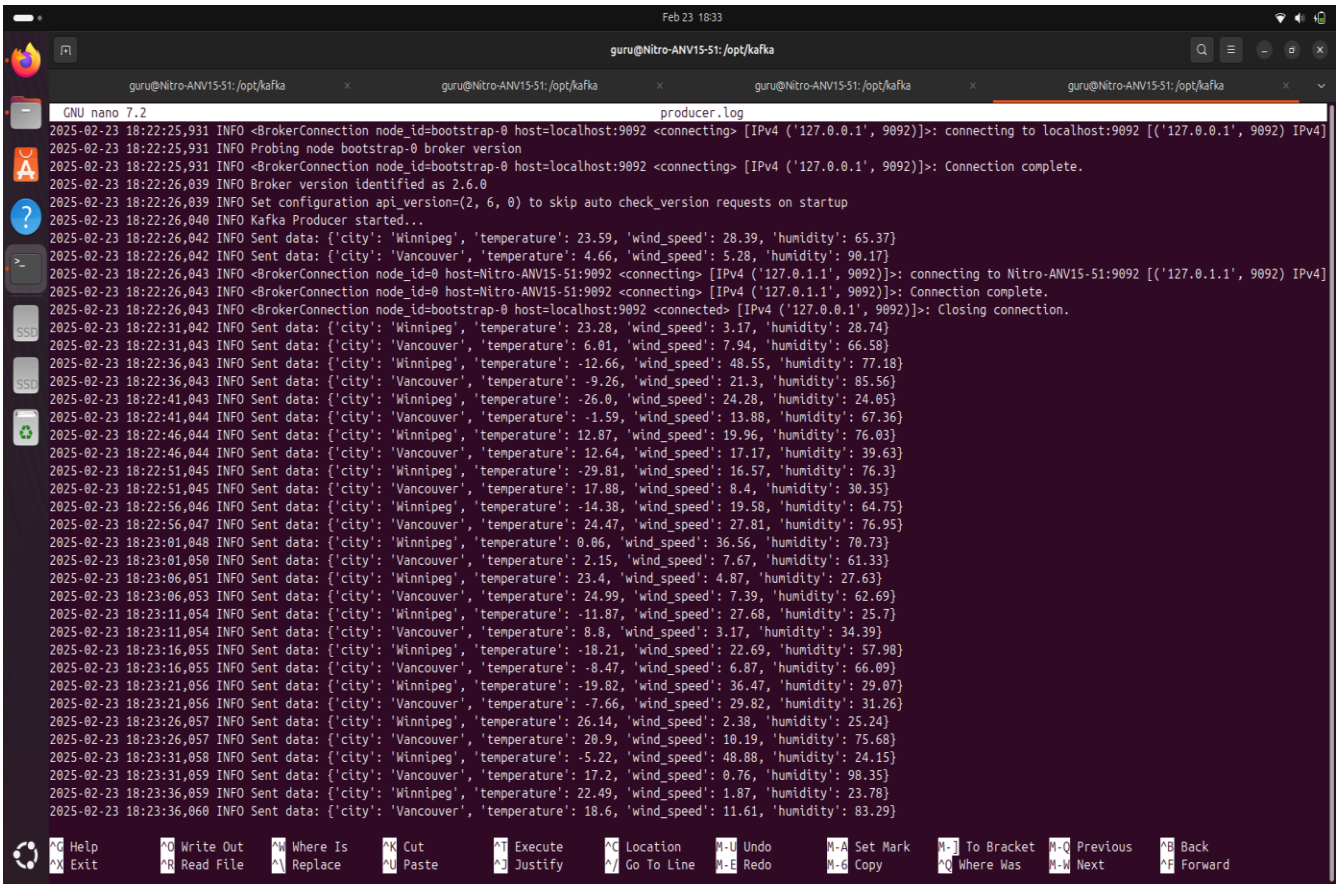
Finally, I:

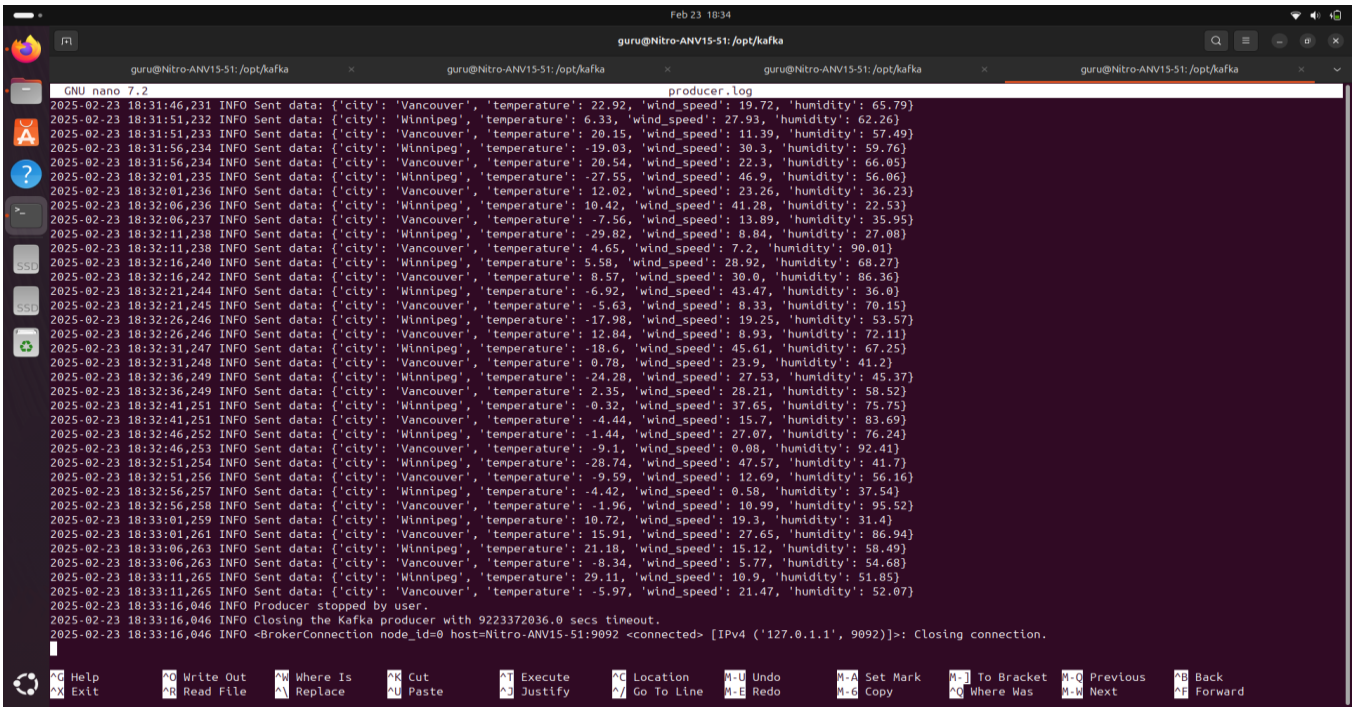
1. Stopped the consumer (Ctrl + C).
2. Stopped Kafka.
3. Stopped Zookeeper.

This concluded the assignment steps.

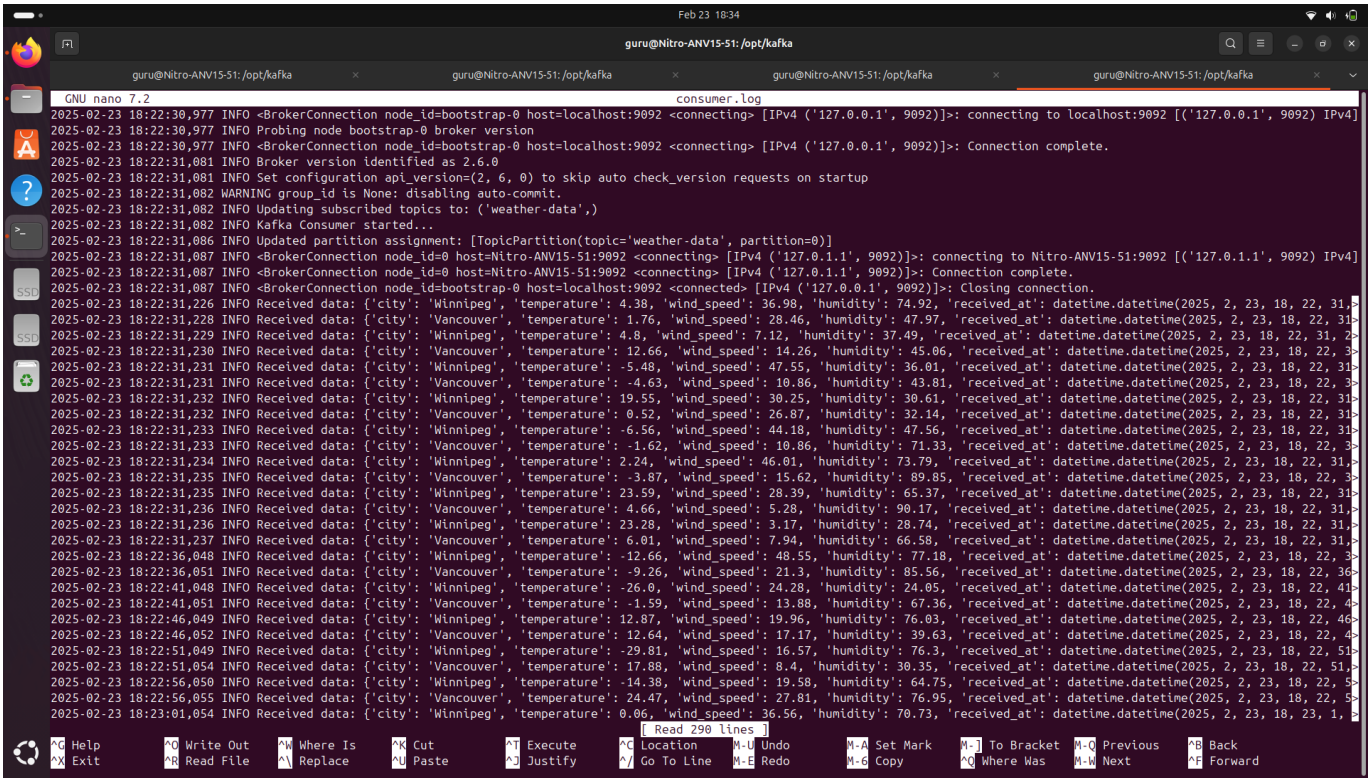
3. Results & Observations

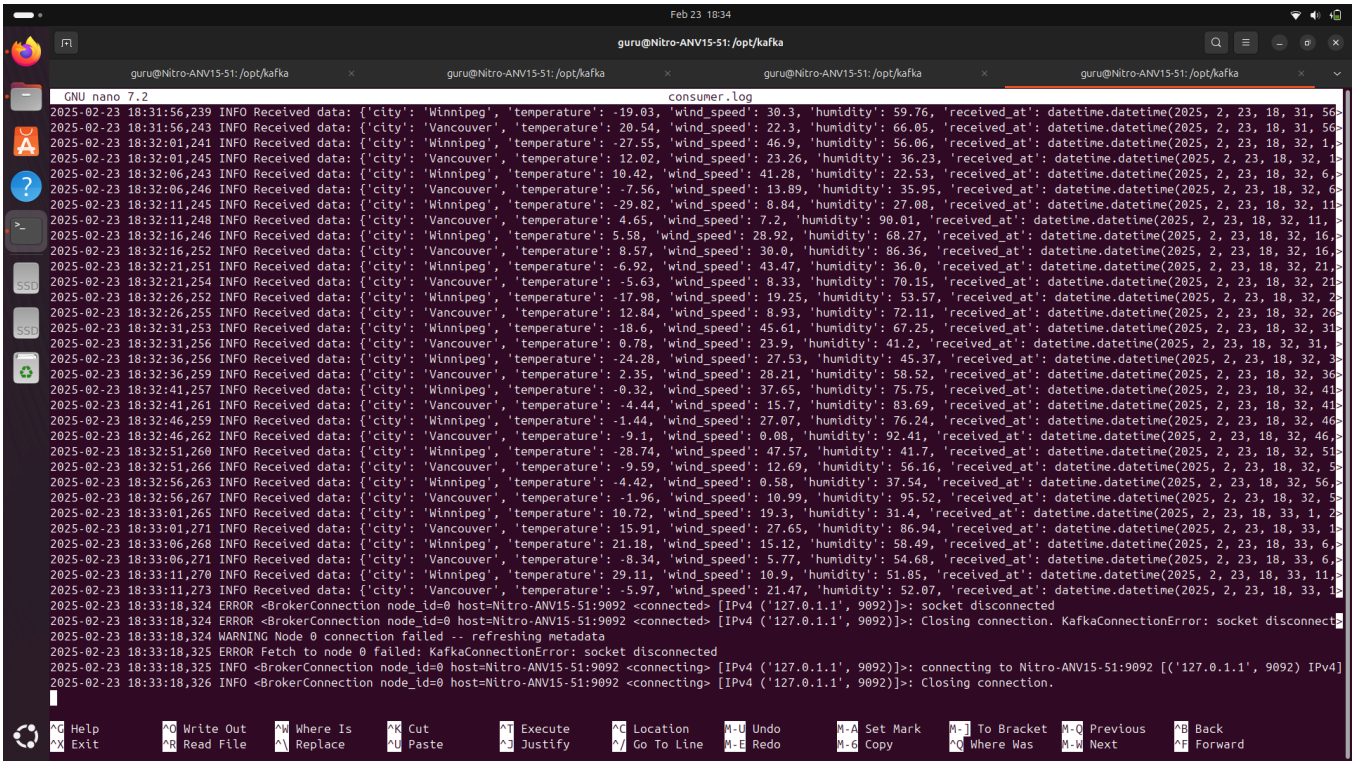
- **Producer Logs:** Confirmed that weather data messages were being generated every five seconds.



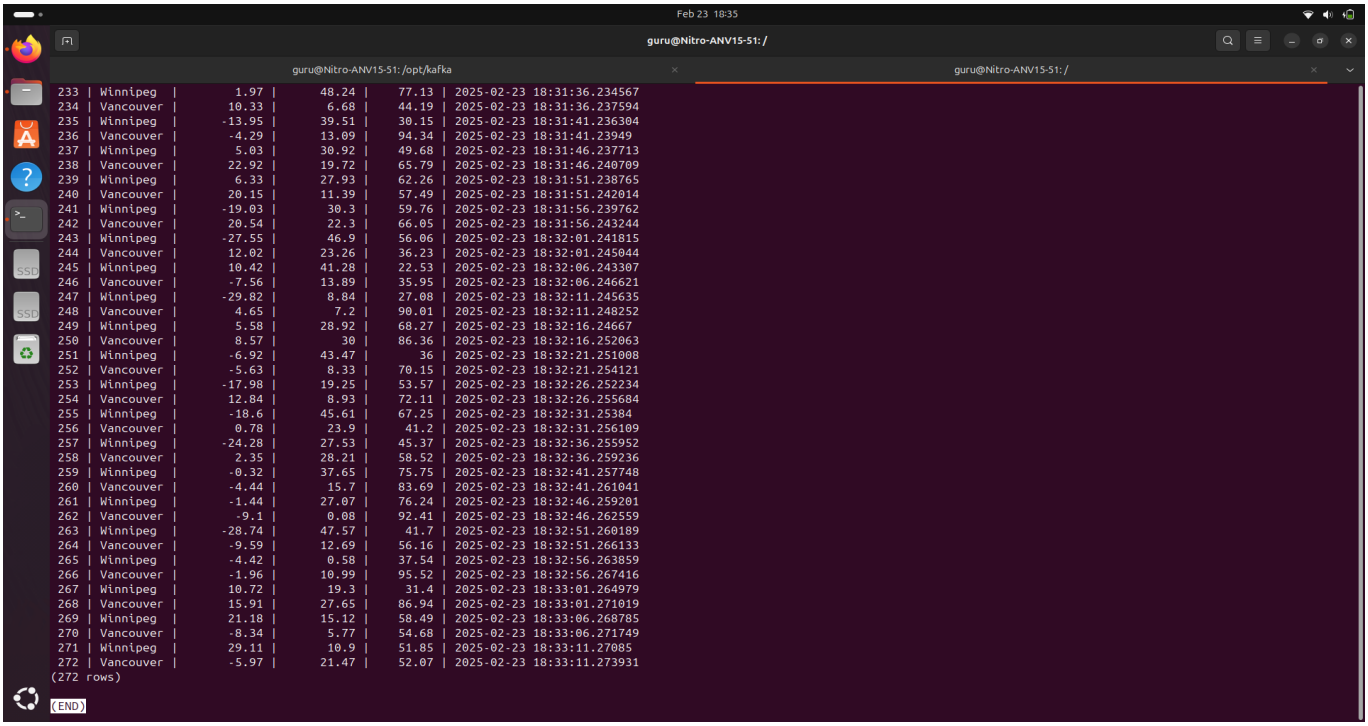


- **Consumer Logs:** Confirmed that messages were consumed and inserted into the database, with timestamps appended.





- **Database Table:** Observed rows in weather_events for both Winnipeg and Vancouver, verifying that the pipeline worked from end to end.



4. Conclusion

This assignment demonstrated how to build a **complete streaming data pipeline** with Apache Kafka. By creating a **Kafka Producer** to generate and send synthetic weather data and a **Kafka Consumer** to receive, transform, and store that data in **PostgreSQL**, we achieved real-time data flow. The logged events and database queries confirm the solution meets all specified requirements.

For a production environment, additional refinements could be made, such as scaling topics, adding partitions for parallel consumption, or applying further data transformations. However, for this assignment, all objectives have been met.